Norwegian University of Science and Technology
Department of Computer and Information Science

Final Exam in
TDT4125 Algorithm Construction, advanced course

Saturday May 24, 2014
Time: 09:00–13:00

Aids: (B) All printed/handwritten; specific, simple calculator

Language: English

Please read the entire exam before you start, plan your time, and prepare any questions for when the teacher comes to the exam room. Make assumptions where necessary. Keep your answers short and concise. Long explanations that do not directly answer the questions are given no weight.

All subtasks count equally toward the final score.

Problem 1

a) Briefly discuss the idea of a relaxation, as it applies to the set of feasible solutions. How could the same concept be applied to what consistutes a valid instance of a problem? What would the difference be? How does this latter form of relaxation relate to reductions?

Solution: Relaxing the constraints on solutions means that the solution set of the original is a subset of our new set, so our solution will be at least optimal. This will generally make the problem easier to solve, but won't necessarily give us a valid solution to the original problem.

If, however, we relax the set of input instances, we are left with a greater set of instances we need to deal with, which cannot make the problem easier—but potentially harder. This would be the case if we moved from metric TSP to general TSP, for example.

If you relax the constraints on problem A and end up with problem B, you have a reduction from A to B, as any instance of A can be solved by B.

b) How is the idea of relaxation used when constructing approximation algorithms?

Solution: The relaxation is at least optimal, but not necessarily feasible. If we can construct a feasible solution that is within a factor of k of the solution to the relaxed problem, it will be within a factor of k of the optimum as well.

c) When dealing with linear programming, a straightforward approach to relaxation can sometimes lead to a loss of too much structure or information. Discuss briefly a common way to mitigate this, and to retain some information about what has been removed, but in a less strict manner.

Solution: What we're going for here is Lagrangean relaxation, in which constraints are not simply dropped, but replaced by penalties in the objective function. Thus, breaking those constraints is no longer forbidden, but it will generally lead to worse objective values.

d) How can one safely remove the randomness from a randomized approxiomation algorithm?

Solution: One can use a process of derandomization by setting each variable in sequence so as to maximize the expected value of the remainder. One will then have preserved the guarantees given by the expected value, but the algorithm is now deterministic.

e) Assume that you want to minimize the objective $\max_{i=1\ldots n} c_i x_i$, where the coefficients c_i are all different, and are all positive integers, and you have the constraint $x_i \in \{0, 1\}$, for $i = 1 \ldots n$. Show how you could reduce this to the objective of minimizing $\sum_{i=1}^{n} c'_i x_i$,

with a different set of coefficients c', so that the resulting solution for x will be optimal for your original problem, regardless of other constraints.

Solution: Simply let $c'_i = 2^{c_i}$. If you now solve the problem for the linear function, you will also get the min-max solution. Assume that this is not the case—that there was some solution that had a lower maximum. This would mean removing the highest power of two from the sum, which would necessarily lead to a lower sum, no matter which of the lower powers were included. This is a contradiction.

Consider the following integer programming formulation of the traveling salesman problem. The graph $G = (V, E)$ has nodes corresponding to the cities, and edges corresponding to the distances between them. Let $\delta(S)$ be the subset of edges in E that have exactly one endpoint in S. The variable x_e in the integer formulation is 1 if the edge is part of the tour, and 0 otherwise.

$$\begin{aligned} \min \quad & \sum_{e \in E} c_e x_e \\ \text{s.t.} \quad & \sum_{e \in \delta(S)} x_e \geq 2\,, \quad \forall S \subset V, S \neq \emptyset\,, \\ & x_e \in \{0,1\}\,, \quad \forall e \in E\,. \end{aligned}$$

f) Though you have no guarantees for the quality of the results, you would like to run some experiments using a linear programming relaxation of the program. Explain how you would do this.

Solution: First, replace $x_e \in \{0,1\}$ by $0 \leq x_e \leq 1$, and you have the relaxation. The problem is that you have an exponential number of constraints. You can solve this with the ellipsoid method, for which you need a separation oracle. (Note that we can easily represent the contraints with a bounded number of bits.) One possible such oracle is a simple traversal, using the edges e for which $x_e = 1$, starting in any node. If your traversal leads to a tour (visiting each node once, returning where you started), the solution is feasible. Otherwise, it is not feasible, and you need to return a broken constraint.

If you did not reach all nodes at all, you can simply let S be the set of nodes you reached, and you have a broken constraint. If you reached all nodes, but your traversal was not a tour, at least one node will have just one edge incident to it in your traversal. Delete it and return the resulting set of nodes as a broken constraint.

The *min-max TSP problem* is a version of the TSP problem where we do not wish to minimize the *sum* of the distances in the tour, but rather we wish to minimize the magnitude of the greatest of those distances. In other words, we wish to minimize

$$\max\{c_{k(n)k(1)}, c_{k(1)k(2)}, \ldots, c_{k(n-1)k(n)}\}\,,$$

where c_{ij} is the distance between cities i and j and $k(i)$ is the ith city in the tour.

g) Rewrite the integer program for TSP so that it is an integer program for the min-max TSP problem. Note that the method from d) will not work here.

Solution: For example, introduce a new variable y for the maximum edge of the solution.

$$\begin{aligned} \min \quad & y \\ \text{s.t.} \quad & y \geq c_e x_e\,, && \forall e \in E \\ & \sum_{e \in \delta(S)} x_e \geq 2\,, && \forall S \subset V, S \neq \emptyset\,, \\ & x_e \in \{0,1\}\,, && \forall e \in E\,. \end{aligned}$$

h) Show that min-max TSP is NP-hard, even if our cost function c is metric, that is, $c_{ik} \leq c_{ij} + c_{jk}$.

Solution: Simple reduction from Hamilton-cycle (similar to plain metric TSP). Simply let all distances be 1 or 2, assigning 1 to those from the original graph. If we find a tour whose longest edge is 1, we have a Hamilton cycle.

i) What approximation bound would the double-tree algorithm give for the min-max TSP problem for metric inputs? Explain your reasoning clearly and thoroughly.

Solution: The spanning tree would still be at least optimal (i.e., its max-edge would be no greater than that of an optimum tour). This is because if some other spanning tree had a smaller max-edge, at least one of the edges of that tree would form a cycle in our tree including our max-edge. We could replace our max-edge with the cycle-forming edge from the other tree (which must necessarily be smaller), and thus have a better solution, which is a contradiction.

The problem is the short-cutting. When replacing c_{ij}, c_{jk} with c_{ik}, we know that $c_{ik} \leq c_{ij} + c_{jk}$, which is good when we're after a sum. In our case, though, we're replacing the cost $\max\{c_{ij}, c_{jk}\}$. In general replacing replacing m edges with a single could give us a cost that is increased by a factor of m. In the worst case (e.g., if the nodes are in a line) we would need to short-cut $n-1$ edges. So if our spanning tree has a cost of C, our Eulerian graph would still have a cost of C, but our short-cutted Eulerian graph could now have a cost of $\max\{C, (n-1)C\} = (n-1)C$. In other words, our approximation is, in the worst case, $(n-1)\text{OPT}$.

Now consider the case of so-called *ultrametric* distances. This means that our costs now obey the so-called *strong* triangle inequality:

$$c_{ik} \leq \max\{c_{ij}, c_{jk}\}$$

Assume that you modify Christofides' algorithm for the min-max problem in the following way: You use some given black-box procedure that can efficiently find both a min-max spanning tree (a spanning tree whose maximum edge is minimum) and a min-max matching (a matching whose maximum edge is minimum). You now use these instead of the MST and min-cost matching subroutines in Christofides' algorithm.

j) What approximation bound would this modification of Christofides' algorithm give for the min-max TSP problem for ultrametric inputs? Explain your reasoning clearly and thoroughly.

Solution: The superoptimality arguments are similar to the original. Because the optimal solution contains a spanning tree, a min-max spanning tree must be at least as good as an optimal solution. For the matching we won't do as well as in the original, though. The optimum will contain two matchings that are complements of each other, but we can no longer say that at least one of them is less than or equal to half the optimum, as we're not working with a sum. All we can say, is that both of them must have a maximum that is no more than the optimum, which means that our min-max matching is just guaranteed to be optimal.

Again, *if* we were working with a sum, this would give us an approximation factor of 2 for the (infeasible) Eulerian graph. However, for the min-max problem, we can simply note that the maxima of the two components individually are superoptimal, so the maximum of them will be, too. In other words, at this point, we have a superoptimal, unfeasible solution.

Now, for the short-cutting. The ultrametric is stricter than the metric, which means that our short-cutting is no longer problematic. A short-cut cannot degrade the situation, and we end up with an optimal, feasible solution.